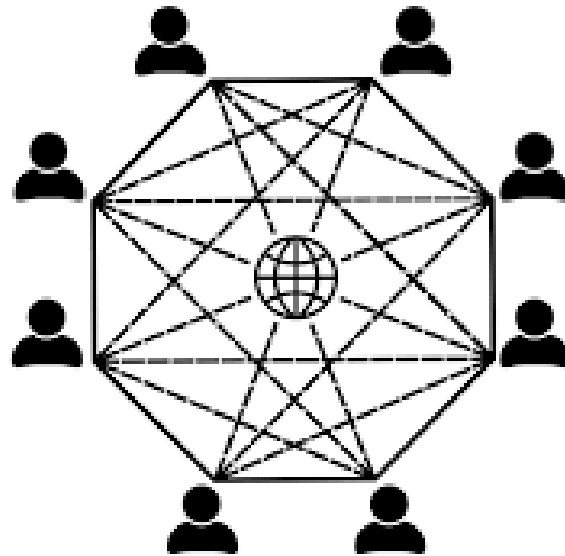
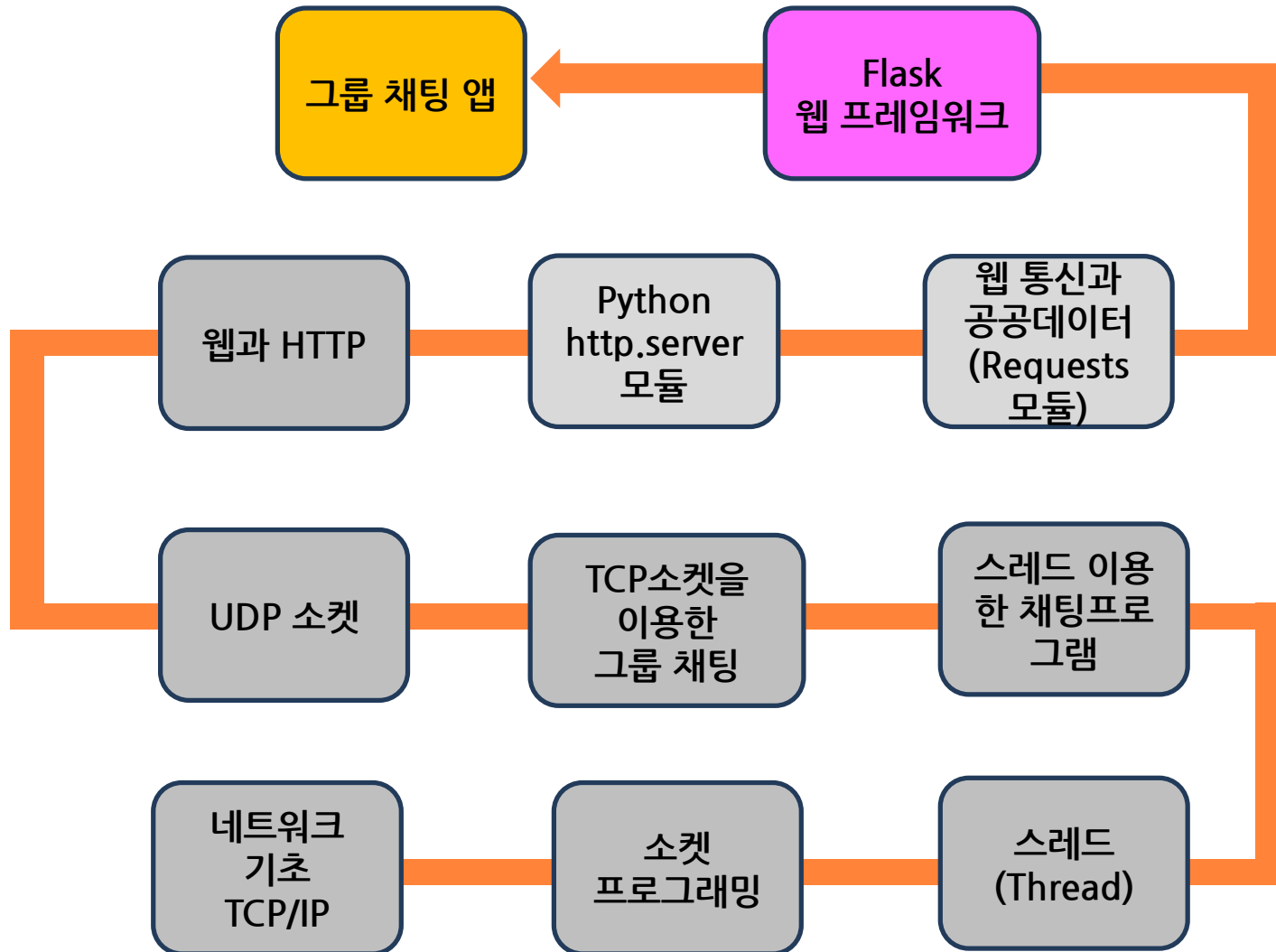


네트워크설계

13주차 - 파이썬 Flask 웹 프레임워크



13주차는 Python web Framework인 Flask를 사용해 봅시다



앱 제작 실습 병행

12주차 피드백) 파이썬 requests 모듈 사용하기

■ Python requests 모듈

- requests 모듈은 Python용 HTTP 라이브러리

- HTTP 요청(Request)을 보내고 응답(Response)을 받는데 사용되는 라이브러리
- 특정 웹사이트에 접속하여 페이지의 HTML 소스코드를 받아서 필요한 정보를 추출할 때 사용



12주차 피드백) 파이썬 requests 모듈 사용하기

■ Python requests 모듈 사용하기

- requests 모듈 제공 API
 - 각 요청방식(Method)에 맞는 함수 제공

GET 방식 – requests.get()

POST 방식 – requests.post()

PUT 방식 – requests.put()

DELETE 방식 – requests.delete()

```
# requests 모듈 사용하기

import requests

requests.get('https://google.com')
```

12주차 피드백) 파이썬 requests 모듈 기본 사용법

■ Python requests 모듈 사용하기 - GET 방식

1) GET 요청 보내고 응답 받기 : `requests.get(url)` 요청하기

```
# Python requests 모듈 사용하기
# HTTP 요청(Request)을 보내고 응답(Response)을 받기위한 라이브러리
import requests, json
```

```
# GET 요청보내고 응답받기
# requests.get(url)
url = 'https://httpbin.org/get'
```

```
response = requests.get(url)
# 응답메시지 출력(JSON type)
print(response.text)
# 응답코드만 출력
print(response.status_code)
# 응답헤더만 출력
print(response.headers)
# 헤더중 Content-Type만 출력
print(response.headers['Content-Type'])
```

```
{
  "args": {},
  "headers": {
    "Accept": "*/*",
    "Accept-Encoding": "gzip, deflate, br, zstd",
    "Host": "httpbin.org",
    "User-Agent": "python-requests/2.32.3",
    "X-Amzn-Trace-Id": "Root=1-6a068622-0951a8f35d4ba46b0a14ea85"
  },
  "origin": "175.212.188.154",
  "url": "https://httpbin.org/get"
}
```

200

```
{'Date': 'Fri, 15 May 2026 02:34:10 GMT', 'Content-Type': 'application/json', 'Content-Length': '319', 'Connection': 'keep-alive', 'Server': 'gunicorn/19.9.0', 'Access-Control-Allow-Origin': '*', 'Access-Control-Allow-Credentials': 'true'}
application/json
```

12주차 피드백) 파이썬 requests 모듈 기본 사용법

■ Python requests 모듈 사용하기 - GET 방식

2) GET요청으로 받은 응답메시지 바디에서 원하는 정보만 처리

- 받은 JSON 파일(혹은 다른 형식의 파일)을 딕셔너리로 변환 후 원하는 정보 추출

```
# 응답메시지 JSON -> 딕셔너리로 변환
```

```
dic_data = response.json()
```

```
print(dic_data)
```

```
# 딕셔너리에서 'url'만 출력하기
```

```
print(dic_data['url'])
```

```
{'args': {}, 'headers': {'Accept': '*/*', 'Accept-Encoding': 'gzip, deflate, br, zstd', 'Host': 'httpbin.org', 'User-Agent': 'python-requests/2.32.3', 'X-Amzn-Trace-Id': 'Root=1-6a0692cb-2fbb944f55304efa695baf98'}, 'origin': '175.212.188.154', 'url': 'https://httpbin.org/get'}
```

```
https://httpbin.org/get
```

12주차 피드백) 파이썬 requests 모듈 기본 사용법

■ Python requests 모듈 사용하기 - GET 방식

3) GET 요청에 파라미터 추가하기 : `requests.get(url, params)`

- 파라미터는 URL의 쿼리 스트링으로 전송 (url + ?query 스트링)

```
# GET 요청시 파라미터 추가하기
# requests.get(url, params)

import requests, json

url = 'https://httpbin.org/get'
# 파라미터 정의(key:value)
params = {"name": "Hello", "data": "World"}

# GET 요청시 파라미터 추가- requests.get(url, params)
response = requests.get(url, params=params)
dic_data = response.json() # JSON -> Dic
print("GET요청 URL:", dic_data['url'])
```

GET요청시
파라미터 데이터는
URL의 쿼리스트링에
실려 전달

GET요청 URL: <https://httpbin.org/get?name=Hello&data=World>

query 스트링

12주차 피드백) 파이썬 requests 모듈 기본 사용법

■ Python requests 모듈 사용하기 - POST 방식

4) POST 요청하고 응답받기 : `requests.post(url, data)`

```
- # POST 요청시 파라미터 추가하기
# requests.post(url, data)

import requests, json

url = 'https://httpbin.org/post'
# 요청메시지의 바디 정의(일반 폼 데이터 형식)
body_data = {"name":"Hello", "data":"World"}

# POST 요청으로 바디를 보내고 응답받기
# requests.post(url, body_data)
response = requests.post(url, data=body_data)
dic_data = response.json() # JSON -> Dic
print("POST 요청 URL:", dic_data['url'])
print("POST 요청 바디:", dic_data['form'])
```

POST 요청 시
파라미터 데이터는
요청메시지 바디에
실려 전달

```
POST 요청 URL: https://httpbin.org/post
POST 요청 바디: {'data': 'World', 'name': 'Hello'}
```


1. Python **Flask** Web Framework

■ Flask 기초

- Flask는 파이썬 웹 프레임워크
 - Flask는 파이썬을 기반으로 한 마이크로 웹 프레임워크
 - django는 파이썬을 기반으로 한 풀 웹 프레임워크
- 웹 프레임워크(Framework: 틀, 도구상자)란?
 - 웹 사이트나 웹 애플리케이션을 개발할 때 기본적인 구조와 기능들을 제공하여 개발자가 쉽고 빠르게 웹사이트를 개발할 수 있도록 돕는 도구
- Flask 설치

pip install flask

1. Python **Flask** Web Framework

■ Flask 기초 - 공식 매뉴얼

- Flask 공식 매뉴얼

- <https://flask.palletsprojects.com/en/stable/>

Quickstart

A Minimal Application

A minimal Flask application looks something like this:

```
from flask import Flask

app = Flask(__name__)

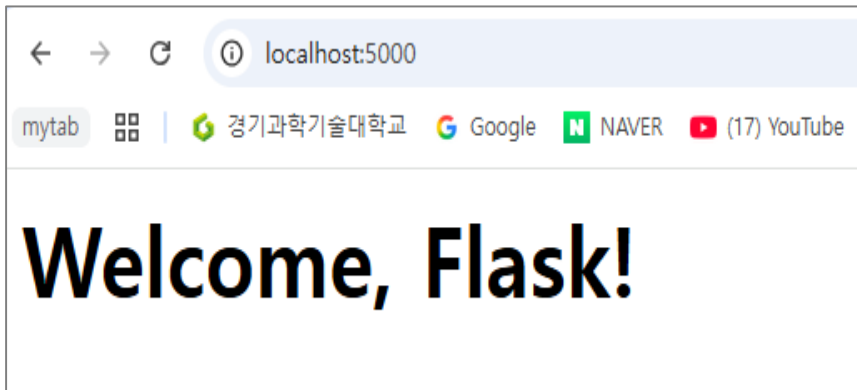
@app.route("/")
def hello_world():
    return "<p>Hello, World!</p>"
```

1. Python Flask Web Framework

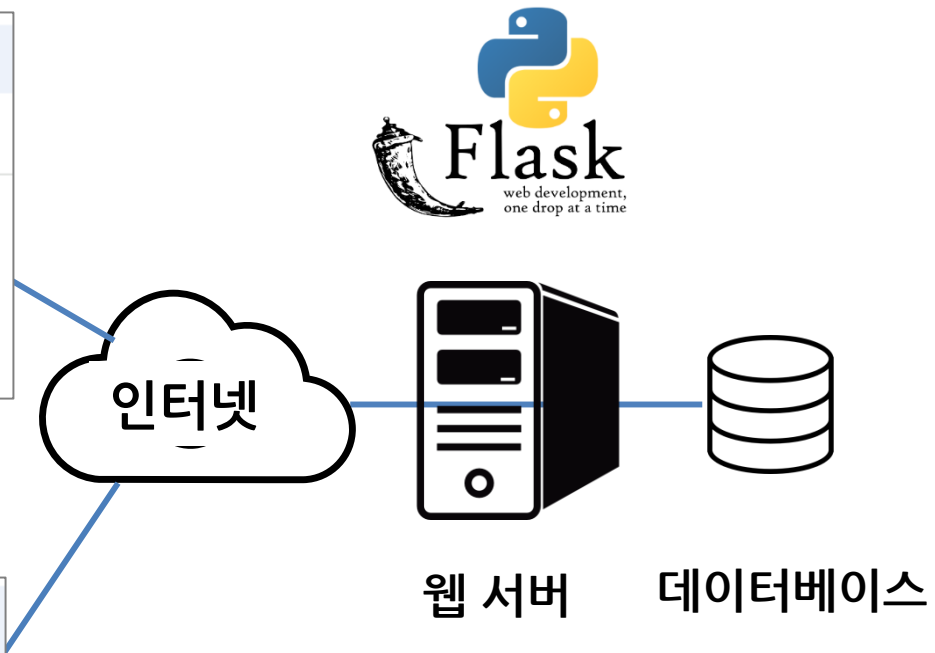
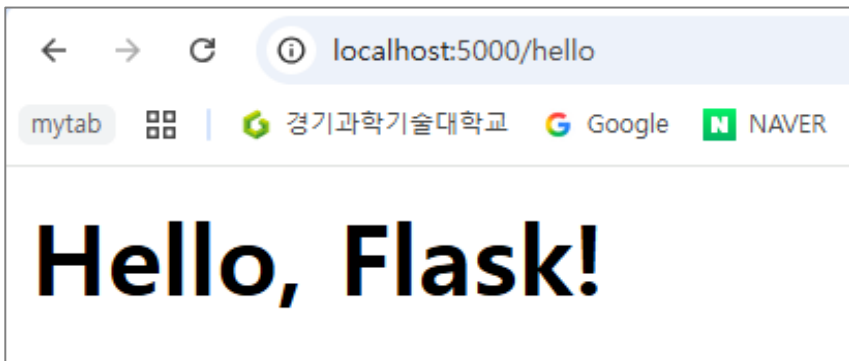
■ 실습1) Flask 기초 : 웹 브라우저 연동

- 웹 브라우저 접속하기

- `http://localhost:5000`



- `http://localhost:5000/hello`

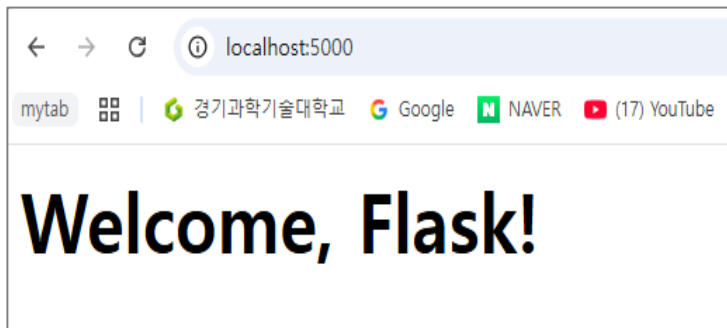


1. Python Flask Web Framework

■ 실습1) Flask 기초 : 기본 코딩(1)

- Flask 애플리케이션을 생성하기 위한 기본 코드 (루트경로: http://localhost:5000)
: flask 객체 생성 → 라우팅 설정 → 서버 실행

```
1 # Flask 웹 프레임워크 기본 사용법
2 from flask import Flask
3
4 # 1. Flask 웹 서버 객체 생성
5 # 현재 파일을 기준으로 Flask 웹 서버 객체를 만들어서 app 변수에 저장
6 app = Flask(__name__)
7
8 # 2. 라우팅 설정: 어떤 주소(경로)로 들어왔을 때, 어떤 함수를 실행할지 연결해 주는 것
9 # 사용자가 홈페이지 접속('/')했을 때 실행할 함수 지정
10 @app.route('/')
11 def index():
12     return "<h1>Welcome, Flask!</h1>"
13
14 # 3. 서버 실행 (로컬 호스트에서 실행)
15 # flask 기본포트는 5000번
16 if __name__ == '__main__':
17     app.run(host='localhost', port=5000)
```

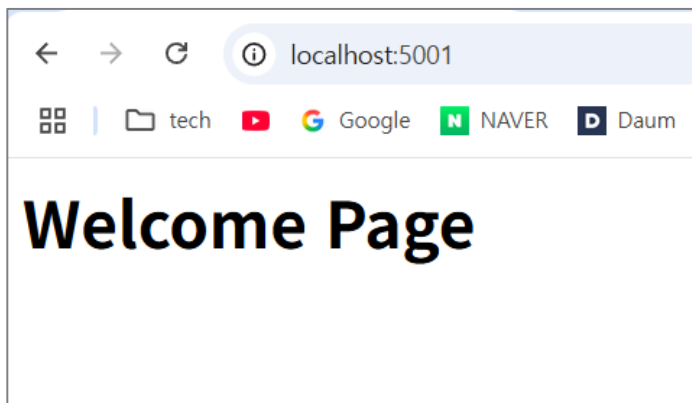


1. Python Flask Web Framework 기초

■ 실습1) Flask 기초 : 라우팅

- Flask 라우팅: 사용자가 요청한 URL 경로에 따라 수행할 함수를 연결해 주는 기능
- “이 주소로 요청이 들어온 경우 밑에 있는 함수를 통해 이 화면을 보여주자!”

```
@app.route('/')
```

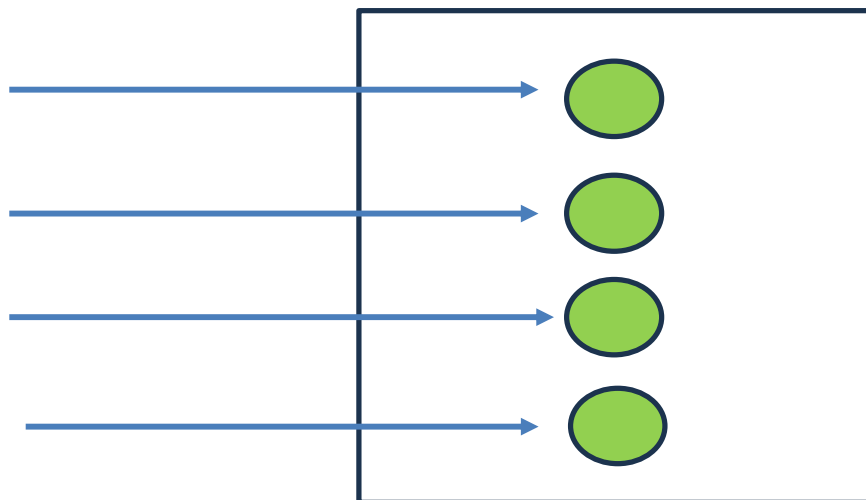


http://localhost:5000

http://localhost:5000/read

http://localhost:5000/create

http://localhost:5000/update

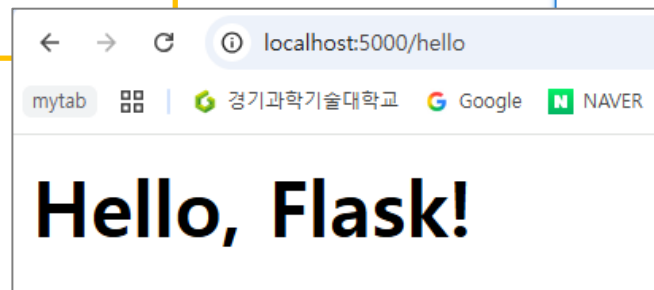


1. Python Flask Web Framework

■ 실습1) Flask 기초 : 기본 코딩(2)

- 라우팅 추가하기 (경로: <http://localhost:5000/hello>)

```
1 # Flask 웹 프레임워크 기본 사용법
2 from flask import Flask
3
4 # 1. Flask 웹 서버 객체 생성
5 # 현재 파일을 기준으로 Flask 웹 서버 객체를 만들어서 app 변수에 저장
6 app = Flask(__name__)
7
8 # 2. 라우팅 설정: 어떤 주소(경로)로 들어왔을 때, 어떤 함수를 실행할지 연결해 주는 것
9 # 사용자가 홈페이지 접속('/')했을 때 실행할 함수 지정
10 @app.route('/')
11 def index():
12     return "<h1>Welcome, Flask!</h1>"
13
14 ## 사용자가 경로 ('/hello')에 접속했을 때 실행할 함수 지정
15 @app.route('/hello/')
16 def hello():
17     return "<h1>Hello, Flask!</h1>"
18
19 # 3. 서버 실행 (로컬 호스트에서 실행)
20 # flask 기본포트는 5000번
21 if __name__ == '__main__':
22     app.run(host='localhost', port=5000)
```



1. Python Flask Web Framework

■ 실습1) Flask 기초 : 기본 코딩(3)

- 라우팅 경로에 <변수명> 형식으로 처리하기 (경로: <http://localhost:5000/hello/name>)

예) <http://localhost:5000/hello/name>

- hello 뒤에 오는 이름을 변수로 받아서 동적으로 웹 페이지를 제공해 봅시다.

기존 코드에 추가해 봅시다.

- 라우팅에서 <변수명>로 처리하고, 함수에서 인자로 받아오기

```
19 # 4. 라우팅 경로에 <변수명> 사용하기
20 @app.route('/hello/<name>/') # URL 경로에 <변수명> 추가
21 def sayHello(name):          # 변수명을 함수의 인자로 전달
22     # 동적으로 웹 페이지 변경
23     return f"<h1>Hello, {name}님<br><br>Flask의 세계에 오신것을 환영합니다!</h1>"
```

← → ↻ ⓘ localhost:5000/hello/홍길동/

Hello, 홍길동님

Flask의 세계에 오신것을 환영합니다!

2. Python Flask GET, POST 메소드 처리

■ 실습2) Flask로 GET, POST 요청 처리하기

- HTTP 요청 메소드인 GET, POST로 전달된 데이터를 Flask에서 처리하는 방법

```
1 # Flask로 GET, POST 요청 처리하기
2 from flask import Flask, request
3
4 # flask 객체 생성
5 app = Flask(__name__)
6
7 # [홈 화면] 이름을 입력할 수 있는 form 화면 보여주기
8 html_form = """
9     <!DOCTYPE html>
10     <html>
11         <body>
12             <h1>Flask로 GET vs POST 요청 처리하기</h1>
13             <p>두 가지 방식으로 이름을 서버로 보내고, 주소창(URL) 비교해 보기.</p>
14             <hr>
15
16             <h3> GET 방식으로 파라미터(이름) 보내기</h3>
17             <form action="/get" method="GET">
18                 이름: <input type="text" name="user_name" required> <br><br>
19                 <input type="submit" value="GET 전송">
20             </form>
21
22             <br><hr><br>
23
24             <h3> POST 방식으로 파라미터(이름) 보내기</h3>
25             <form action="/post" method="POST">
26                 아이디: <input type="text" name="user_name" required> <br><br>
27                 <input type="submit" value="POST 전송">
28             </form>
29         </body>
30     </html>
31     """
```

해당 코드는 공지사항에
올려져 있습니다

URL- http://localhost:5000

홈 화면의 폼 페이지

Flask로 GET vs POST 요청 처리하기

두 가지 방식으로 이름을 서버로 보내고, 주소창(URL) 비교해 보기.

GET 방식으로 파라미터(이름) 보내기

이름:

GET 전송

POST 방식으로 파라미터(이름) 보내기

아이디:

POST 전송

2. Python Flask GET, POST 메소드 처리

■ 실습2) Flask GET, POST 메소드 처리

- GET 요청 : 모든 파라미터를 URL의 쿼리스트링으로 보내 요청하는 방식
“http://localhost:5000/get/?user_name=홍길동”
- Flask는 기본적으로 GET방식으로 동작
- 쿼리스트링에서 key의 value를 가져오려면, request.args.get(key)

```
33 # 라우팅 설정
34 # 루트('/')로 접속시 홈 화면 전송하기
35 @app.route('/')
36 def index():
37     return html_form
38
39 # 경로('/get')으로 GET방식 요청시 처리
40 @app.route('/get/', methods=['GET'])
41 def hello_get():
42     # URL 쿼리스트링에서(?user_name=value) value 데이터(이름) 가져오기
43     name = request.args.get('user_name')
44     return f'''
45         <h3>[GET 요청 처리화면]</h3>
46         <h1>안녕하세요, {name}님!</h1>
47         <p>주소창(URL)에 입력한 이름이 그대로 보입니다</p>
48         <a href='/'>돌아가기 </a>
49     '''
50
```

localhost:5000/get/?user_name=홍길동

GET 방식으로 파라미터(이름) 보내기

이름:

GET 전송

[GET 요청 처리 화면]

안녕하세요, 홍길동님!

주소창(URL)에 입력한 이름이 그대로 보입니다

[돌아가기](#)

2. Python Flask GET, POST 메소드 처리

■ 실습2) Flask GET, POST 메소드 처리

- POST 요청: 전달하려는 정보가 HTTP 요청 body에 포함되어 전달
 - 전달하려는 정보를 body(Form Data)에 실어서 post 방식으로 전달하는 방법

{'user_name':'홍길동'}이 응답 body에 실려서 전달

- Flask로 POST 처리하려면 라우팅에서 POST를 추가해야 함. (추가하지 않으면 404 Not found!)
- 요청메시지 body에서 key의 value를 가져오려면, `request.form.get(key)`

```
51 # 경로('/post')으로 POST방식 요청시 처리
52 # flask는 기본적으로 GET방식으로 처리. POST방식 처리하려면
53 @app.route('/post/', methods=['POST'])
54 def hello_post():          POST 추가
55     # 요청메시지 body(form 데이터)에서 value 데이터(이름) 가져오기
56     name = request.form.get('user_name')
57     return f'''
58         <h3>[GET 요청 처리화면]</h3>
59         <h1>안녕하세요, {name}님!</h1>
60         <p>주소창(URL)에 입력한 이름이 그대로 보입니다</p>
61         <a href='/'>돌아가기 </a>
62     '''
63 # flask 웹 서버 실행
64 if __name__ == "__main__":
65     app.run(host="localhost", port=5000)
```

POST 방식으로 파라미터(이름) 보내기

아이디:

POST 전송

[POST 요청 처리 화면]

안녕하세요, 홍길동님!

주소창(URL)에 이름은 보이지 않고, 바디에 안전하게 담아서 보냅니다

[돌아가기](#)

3. Python Flask 응용실습

■ 실습3) Flask 웹 서버 응용실습 : GET과 POST방식으로 로그인하기

- Flask로 웹서버를 구현하고 GET과 POST 요청에 대한 처리방식을 이해하셨다면 다음을 flask로 구현해 보세요.

홈 화면

Flask GET vs POST 로그인 실습

두 가지 방식으로 로그인을 해보고, 주소창(URL) 비교해 보기.

테스트 계정 -> 아이디: admin / 비밀번호: 1234

http://localhost:5000/get?user_id=admin&user_pw=1234

GET 방식으로 로그인 하기

아이디:

비밀번호:

GET 로그인

GET 방식으로 로그인 하기

아이디:

비밀번호:

GET 로그인

[GET 방식 결과 페이지]

로그인 성공! admin님 환영합니다!.

[돌아가기](#)

POST 방식으로 로그인 하기

아이디:

비밀번호:

POST 로그인

POST 방식으로 로그인 하기

아이디:

비밀번호:

POST 로그인

<http://localhost:5000/post>

[POST 방식 결과 페이지]

로그인 성공! admin님 환영합니다!

[돌아가기](#)

3. Python Flask 응용실습

■ 실습3) Flask 웹 서버 응용실습 : GET과 POST방식으로 로그인하기

```
1 # Flask 웹 서버에 GET/POST방식으로 로그인하기
2 from flask import Flask, request
3 # Flask 웹서버 객체 생성
4 app = Flask(__name__)
5
```

```
6 # 테스트용 회원 데이터 (아이디: admin, 비밀번호: 1234)
7 USER_DB = {"id": "admin", "pw": "1234"}
```

회원 DB

```
9 # 로그인 입력 화면(HTML 문자열)
```

```
10 html_form = """
```

```
11     <!DOCTYPE html>
```

```
12     <html>
```

```
13         <body>
```

```
14             <h1>Flask GET vs POST 로그인 실습</h1>
```

```
15             <p>두 가지 방식으로 로그인을 해보고, 주소창(URL) 비교해 보기.</p>
```

```
16             <p>테스트 계정 -> 아이디: admin / 비밀번호: 1234 </p>
```

```
17             <hr>
```

```
18             <h3> GET 방식으로 로그인 하기</h3>
```

```
19                 <form action="/get" method="GET">
```

```
20                     아이디: <input type="text" name="user_id" required> <br><br>
```

```
21                     비밀번호: <input type="password" name="user_pw" required> <br><br>
```

```
22                     <button type="submit">GET 로그인</button>
```

```
23                 </form>
```

```
24             <br><hr><br>
```

```
25
```

```
26
```

```
27             <h3> POST 방식으로 로그인 하기</h3>
```

```
28                 <form action="/post" method="POST">
```

```
29                     아이디: <input type="text" name="user_id" required> <br><br>
```

```
30                     비밀번호: <input type="password" name="user_pw" required> <br><br>
```

```
31                     <button type="submit">POST 로그인</button>
```

```
32                 </form>
```

```
33             </body>
```

```
34         </html>
```

```
35     """
```

해당 코드는 공지사항에
올려져 있습니다

파일: flask_login_이름.py

홈 화면

Flask GET vs POST 로그인 실습

두 가지 방식으로 로그인을 해보고, 주소창(URL) 비교해 보기.

테스트 계정 -> 아이디: admin / 비밀번호: 1234

GET 방식으로 로그인 하기

아이디:

비밀번호:

GET 로그인

POST 방식으로 로그인 하기

아이디:

비밀번호:

POST 로그인

3. Python Flask 응용실습

■ 실습3) Flask 웹 서버 응용실습 : GET과 POST방식으로 로그인하기

- GET방식과 POST방식으로 로그인 기능을 구현해 보세요

```
36
37 @app.route('/')
38 def home():
39     return html_form
40
41 # 1. GET 방식 로그인 처리
42
43 GET방식으로 전달받은 아이디와 비번을
44 추출하여 DB값과 비교해 결과 출력하기
45
46 # 2. POST 방식 로그인 처리
47
48 POST방식으로 전달받은 아이디와 비번을
49 추출하여 DB값과 비교해 결과 출력하기
50
51 if __name__ == '__main__':
52     app.run(host='localhost', port=5000)
```

GET방식) 아이디와 비밀번호가 같은 경우 출력

[GET 방식 결과 페이지]

로그인 성공! admin님 환영합니다!

[돌아가기](#)

GET방식) 아이디와 비밀번호가 다른 경우에 결과 출력

[GET 방식 결과 페이지]

로그인 실패! 아이디나 비밀번호를 확인하세요

[돌아가기](#)

POST방식) 아이디와 비밀번호가 같은 경우 출력

[POST 방식 결과 페이지]

로그인 성공! admin님 환영합니다!

[돌아가기](#)

POST방식) 아이디와 비밀번호가 다른 경우에 결과 출력

[POST 방식 결과 페이지]

로그인 실패! 아이디나 비밀번호를 확인하세요

[돌아가기](#)

4. Python Flask 심화실습

■ 심화실습) Flask 웹 서버 심화실습

- Flask를 이용해 동적으로 웹 페이지를 구현해 봅시다

```
# flask를 이용해 웹 서버 구현하기
from flask import Flask
from flask import request

app = Flask(__name__)

elements = ['웹서버와 웹브라우저']
def html_get():
    liTags = ''
    for element in elements:
        liTags = liTags + f"<li>{element}</li>"
    return f'''<!doctype html>
<html>
    <body>
        <h1> Web의 구성요소 4가지</h1>
        <ul>
            {liTags}
        </ul>
        <h3><a href="/create">웹 구성요소 추가하기</a></h3>
        <h2>Flask의 세계에 오신 것을 환영합니다</h2>
        Flask를 이용해 웹서버를 제작해 봅시다!
    </body>
</html>
'''
```

Web의 구성요소 4가지

- 웹서버와 웹브라우저
- HTML
- HTTP
- URL

웹 구성요소 추가하기

Flask의 세계에 오신 것을 환영합니다

Flask를 이용해 웹서버를 제작해 봅시다!

4. Python Flask 심화실습

■ 심화실습) Flask 웹 서버 심화실습

```
html_post = '''
    <!doctype html>
    <html>
        <body>
            <h1> 웹 구성요소 추가하기</h1>
            <form method="POST" action="/create">
                <input name="item" type="text" placeholder="add new item">
                <input type="submit" value="create">
            </form>
        </body>
    </html>
    '''
```

4. Python Flask 심화실습

■ 심화실습) Flask 웹 서버 심화실습

```
# 라우팅
@app.route('/')
def index():
    return html_get()

@app.route('/create', methods=['GET', 'POST'])
def create():
    if request.method == 'GET':
        return html_post

    elif request.method == 'POST':
        elements.append(request.form['item'])
    return html_get()

app.run(port=5001) # flask default port 5000
```


13주차 실습과제 안내

■ 과제명: 실습3) Flask 웹 프레임워크를 이용한 웹 서버 로그인 기능 구현하기

- 제출파일: flask_login_이름.py 혹은 flask_login_이름.ipynb
- 실습 시 잘 안된 부분 혹은 이해 안된 부분은 comment 남겨주세요.

Flask GET vs POST 로그인 실습

두 가지 방식으로 로그인을 해보고, 주소창(URL) 비교해 보기.

테스트 계정 -> 아이디: admin / 비밀번호: 1234

GET 방식으로 로그인 하기

아이디:

비밀번호:

GET 로그인

POST 방식으로 로그인 하기

아이디:

비밀번호:

POST 로그인

[GET 방식 결과 페이지]

로그인 실패! 아이디나 비밀번호를 확인하세요

[GET 방식 결과 페이지]

로그인 성공! admin님 환영합니다!

[돌아가기](#)

[POST 방식 결과 페이지]

로그인 실패! 아이디나 비밀번호를 확인하세요

[POST 방식 결과 페이지]

로그인 성공! admin님 환영합니다!

[돌아가기](#)



Thank You
